

Cursada II Cuatrimestre 2024

ALGORITMOS Y ESTRUCTURAS DE DATOS

Trabajo Práctico N° 1 – Repaso General

Tipos de datos, Estructuras de Control

Ejercicio 1:

Dada la siguiente información, elija una estructura de datos que permita guardarla adecuadamente.

- Guerra del Peloponeso 431 a.C
- Revolución de Mayo 1810 d.C
- Llegada de los españoles a América 1492 d.C
- Comienzo de la construcción de la gran Muralla China 214 a.C

Explicación:

Un diccionario es ideal porque permite almacenar pares de claves y valores, donde la clave puede ser la descripción del evento y el valor puede ser la fecha asociada.

Respuesta corta:

```
eventos = [  
    {"evento": "Guerra del Peloponeso", "fecha": "431 a.C"},  
    {"evento": "Revolución de Mayo", "fecha": "1810 d.C"},  
    {"evento": "Llegada de los españoles a América", "fecha": "1492 d.C"},  
    {"evento": "Comienzo de la construcción de la gran Muralla China", "fecha":  
"214 a.C"}  
]
```

Respuesta larga:

```
eventos = [  
    {"evento": "Guerra del Peloponeso", "fecha": "431 a.C"},  
    {"evento": "Revolución de Mayo", "fecha": "1810 d.C"},  
    {"evento": "Llegada de los españoles a América", "fecha": "1492 d.C"},  
    {"evento": "Comienzo de la construcción de la gran Muralla China", "fecha":  
"214 a.C"}  
]  
  
def convertir_fecha(fecha):  
    partes = fecha.split()  
    año = int(partes[0])  
    era = partes[1]
```

```

if era == "a.C":
    return -año
elif era == "d.C":
    return año
return año

eventos_ordenados = sorted(eventos, key=lambda e: convertir_fecha(e["fecha"]))
for evento in eventos_ordenados:
    print(f'{evento["evento"]}: {evento["fecha"]}')

```

Ejercicio 2:

Asígneles a las variables **num1** y **num2** los distintos valores indicados en la tabla. Realice las siguientes cuentas con las operaciones indicadas almacenando el resultado en la variable **resul1**. En la tabla que aparece a continuación coloque los resultados obtenidos y justifíquelo.

Valor de num1	Valor de num2	Operación	Valor de resul	Justificación
5	2	//	2	Se trata de una división entera por lo cual el valor de resultado se tratará de un número entero.
7.2	9.8	*	70.56	Se trata de una multiplicación por lo cual el valor de resultado se tratará de una operación de dos números multiplicados entre si.
7	3.1	-	3.9	Se trata de una resta por lo cual el valor de resultado se tratará de una operación de dos números que se sustraen entre si.
10.45	7	+	17.45	Se trata de una suma por lo cual el valor de resultado se tratará de una operación de dos números sumados entre si.
"republica"	"argentina"	+	republicaargentina	Se trata de una suma pero en este caso se concatenan las dos cadenas de "república" y "argentina"

Valor de num1	Valor de num2	Operación	Valor de resul	Justificación
"argentina"	3	*	argentinaargentinaargentina	Se trata de una multiplicación pero en este caso se multiplica la cadena por el valor numérico del num2, concatenando esa cantidad de cadenas

Ejercicio 3:

Escribir un programa que calcule el perímetro de un triángulo. Para ello el usuario deberá ingresar los lados del triángulo.

Respuesta corta:

```
base=float(input("ingrese valor de la base= "))
lado1=float(input("ingrese valor del lado 1= "))
lado2=float(input("ingrese valor del lado 2= "))

perimetro=base+lado1+lado2

print(perimetro)
```

Respuesta larga:

Tenemos que considerar que para formar un triángulo con tres lados, deben cumplirse las siguientes propiedades:

Desigualdad triangular: La suma de las longitudes de dos lados cualesquiera debe ser mayor que la longitud del tercer lado.

Específicamente, para lados (a), (b) y (c):

$$a + b > c$$

$$a + c > b$$

$$b + c > a$$

Longitudes positivas: Todos los lados deben tener longitudes positivas, entonces podríamos corroborar que se cumplan estas propiedades y de no ocurrir lanzar una excepción

```
def calcular_perimetro_triángulo():
    try:
        # Pedir los valores de los lados al usuario
        a = float(input("Ingrese la longitud del lado a: "))
        b = float(input("Ingrese la longitud del lado b: "))
        c = float(input("Ingrese la longitud del lado c: "))

        # Verificar que todos los lados sean positivos
```

```

if a <= 0 or b <= 0 or c <= 0:
    raise ValueError("Todos los lados deben ser positivos.")

# Verificar la desigualdad triangular
if a + b <= c or a + c <= b or b + c <= a:
    raise ValueError("Los lados no forman un triángulo válido según la
desigualdad triangular.")

# Calcular el perímetro si se cumplen las condiciones
perimetro = a + b + c
return f"El perímetro del triángulo es: {perimetro}"

except ValueError as e:
    return f"Error: {str(e)}"

# Ejemplo de uso
resultado = calcular_perimetro_trianguolo()
print(resultado)

```

Ejercicio 4:

Escribir un programa que calcule el área de un triángulo. Para ello el usuario deberá ingresar la base y la altura.

Respuesta corta:

```

# Pedir los valores de la base y la altura al usuario
base = float(input("Ingrese la base del triángulo: "))
altura = float(input("Ingrese la altura del triángulo: "))
# Calcular el área del triángulo
area = (base * altura) / 2

```

Respuesta larga:

```

def calcular_area_trianguolo():
    try:
        # Pedir los valores de la base y la altura al usuario
        base = float(input("Ingrese la base del triángulo: "))
        altura = float(input("Ingrese la altura del triángulo: "))

        # Verificar que la base y la altura sean positivos
        if base <= 0 or altura <= 0:
            raise ValueError("La base y la altura deben ser valores
positivos.")

        # Calcular el área del triángulo
        area = (base * altura) / 2

```

```
        return "El área del triángulo es: " + str(area)

    except ValueError as e:
        return "Error: " + str(e)
```

Aclaración Fijense como el try-catch se hace siempre igual 😊

Ejercicio 5:

Vamos a definir una fracción que está representada por una lista de dos elementos, el numerador y el denominador. Por ejemplo, la fracción $\frac{3}{4}$ sería la lista (3,4).

Función para leer los valores de una fracción

```
def leer_fraccion():
    while True:
        try:
            numerador = int(input("Ingrese el numerador de la fracción: "))
            denominador = int(input("Ingrese el denominador de la fracción: "))
            if denominador == 0:
                print("El denominador no puede ser 0. Inténtelo de nuevo.")
                continue
            return [numerador, denominador]
        except ValueError:
            print("Entrada no válida. Por favor ingrese números enteros.")
```

a) Escribir un programa que sume dos fracciones. Para ello el usuario deberá ingresar los datos de las dos fracciones por teclado.

En realidad sólo vamos a escribir la función que haga el cálculo propuesto

```
def sumar_fracciones(frac1, frac2):
    numerador1, denominador1 = frac1
    numerador2, denominador2 = frac2
    # Encontrar el denominador común
    denominador_comun = denominador1 * denominador2
    numerador_resultado = (numerador1 * denominador2) + (numerador2 *
denominador1)
    return [numerador_resultado, denominador_comun]
```

b) Escribir un programa que reste dos fracciones. Para ello el usuario deberá ingresar los datos de las dos fracciones por teclado.

```
def restar_fracciones(frac1, frac2):
    numerador1, denominador1 = frac1
    numerador2, denominador2 = frac2
    # Encontrar el denominador común
    denominador_comun = denominador1 * denominador2
    numerador_resultado = (numerador1 * denominador2) - (numerador2 *
denominador1)
    return [numerador_resultado, denominador_comun]
```

c) Escribir un programa que multiplique dos fracciones. Para ello el usuario deberá ingresar los datos de las dos fracciones por teclado.

```
def multiplicar_fracciones(frac1, frac2):
    numerador1, denominador1 = frac1
    numerador2, denominador2 = frac2
    numerador_resultado = numerador1 * numerador2
    denominador_resultado = denominador1 * denominador2
    return [numerador_resultado, denominador_resultado]
```

d) Escribir un programa que divida dos fracciones. Para ello el usuario deberá ingresar los datos de las dos fracciones por teclado.

```
def dividir_fracciones(frac1, frac2):
    numerador1, denominador1 = frac1
    numerador2, denominador2 = frac2
    numerador_resultado = numerador1 * denominador2
    denominador_resultado = denominador1 * numerador2
    return [numerador_resultado, denominador_resultado]
```

Ahora, podemos implementar todo si armamos una clase:

```
class Fraccion:
    def __init__(self, numerador, denominador):
        if denominador == 0:
            raise ValueError("El denominador no puede ser 0.")
        self.numerador = numerador
        self.denominador = denominador

    def __str__(self):
        return "{}/{}".format(self.numerador, self.denominador)

    def __add__(self, otra):
        numerador_resultado = (self.numerador * otra.denominador) +
(otra.numerador * self.denominador)
        denominador_resultado = self.denominador * otra.denominador
```

```

        return Fraccion(numerador_resultado, denominador_resultado)

    def __sub__(self, otra):
        numerador_resultado = (self.numerador * otra.denominador) -
(otra.numerador * self.denominador)
        denominador_resultado = self.denominador * otra.denominador
        return Fraccion(numerador_resultado, denominador_resultado)

    def __mul__(self, otra):
        numerador_resultado = self.numerador * otra.numerador
        denominador_resultado = self.denominador * otra.denominador
        return Fraccion(numerador_resultado, denominador_resultado)

    def __truediv__(self, otra):
        if otra.numerador == 0:
            raise ValueError("No se puede dividir por una fracción con
numerador 0.")
        numerador_resultado = self.numerador * otra.denominador
        denominador_resultado = self.denominador * otra.numerador
        return Fraccion(numerador_resultado, denominador_resultado)

```

Ejercicio 6:

Escribir un programa que permita ingresar datos de un Kiosko. Código del producto, nombre del producto, marca del producto, proveedor, precio, stock. La carga de datos finalizará cuando ingrese como código de producto 9999.

Esta resolución es 100% de ustedes:

```

import datetime

inventario = {}

def mostrar_menu():
    print("/// Menú del Kiosko ///")
    print("1. Agregar producto")
    print("2. Eliminar producto")
    print("3. Modificar producto")
    print("4. Ver inventario")
    print("5. Salir")

def validar_codigo(codigo):
    return codigo.isdigit() and 1 <= int(codigo) <= 9999

def validar_texto(texto):
    return texto.isalpha() and len(texto) > 0

def validar_precio(precio):
    try:

```

```

        precio = float(precio)
        return precio > 0
    except ValueError:
        return False

def validar_cajas(cajas):
    try:
        cajas = int(cajas)
        return cajas > 0
    except ValueError:
        return False

def agregar_producto():
    while True:
        codigo = input("Ingresa el código del producto (9999 para finalizar): ").strip()
        if codigo == '9999':
            return
        if not validar_codigo(codigo):
            print("Código inválido. Debe ser un número entre 1 y 9999.")
            continue
        if codigo in inventario:
            print("El producto con este código ya existe.")
            continue

        nombre = input("Ingresa el nombre del producto: ").strip()
        if not validar_texto(nombre):
            print("Nombre inválido. Debe ser solo texto y no puede estar vacío.")
            continue

        marca = input("Ingresa la marca del producto: ").strip()
        if not validar_texto(marca):
            print("Marca inválida. Debe ser solo texto y no puede estar vacío.")
            continue

        proveedor = input("Ingresa el proveedor del producto: ").strip()
        if not validar_texto(proveedor):
            print("Proveedor inválido. Debe ser solo texto y no puede estar vacío.")
            continue

        precio = input("Ingresa el precio del producto: ").strip()
        if not validar_precio(precio):
            print("Precio inválido. Debe ser un número mayor a 0.")
            continue

        cajas = input("Ingresa la cantidad de cajas del producto: ").strip()
        if not validar_cajas(cajas):
            print("Cantidad de cajas inválida. Debe ser un número entero mayor a 0.")
            continue

```

```

        inventario[codigo] = {
            "nombre": nombre,
            "marca": marca,
            "proveedor": proveedor,
            "precio": float(precio),
            "cajas": int(cajas),
            "stock_por_caja": 10
        }
        print("Producto agregado exitosamente.")

def eliminar_producto():
    codigo = input("Ingresa el código del producto a eliminar: ").strip()
    if codigo in inventario:
        del inventario[codigo]
        print("Producto eliminado exitosamente.")
    else:
        print("Producto no encontrado.")

def modificar_producto():
    codigo = input("Ingresa el código del producto a modificar: ").strip()
    if codigo not in inventario:
        print("Producto no encontrado.")
        return

    print("Deja en blanco si no deseas cambiar el valor.")
    nombre = input(f"Nuevo nombre (actual: {inventario[codigo]['nombre']}): ").strip()
    marca = input(f"Nueva marca (actual: {inventario[codigo]['marca']}): ").strip()
    proveedor = input(f"Nuevo proveedor (actual: {inventario[codigo]['proveedor']}): ").strip()
    precio = input(f"Nuevo precio (actual: {inventario[codigo]['precio']}): ").strip()
    cajas = input(f"Nueva cantidad de cajas (actual: {inventario[codigo]['cajas']}): ").strip()

    if nombre:
        inventario[codigo]['nombre'] = nombre
    if marca:
        inventario[codigo]['marca'] = marca
    if proveedor:
        inventario[codigo]['proveedor'] = proveedor
    if precio:
        try:
            inventario[codigo]['precio'] = float(precio)
        except ValueError:
            print("Formato de precio inválido.")
            return
    if cajas:
        try:
            inventario[codigo]['cajas'] = int(cajas)
        except ValueError:

```

```

        print("Formato de cajas inválido.")
        return

    print("Producto modificado exitosamente.")

def ver_inventario():
    if not inventario:
        print("El inventario está vacío.")
        return

    for codigo, datos in inventario.items():
        stock_total = datos['cajas'] * datos['stock_por_caja']
        print(f"Código: {codigo}")
        print(f"Nombre: {datos['nombre']}")
        print(f"Marca: {datos['marca']}")
        print(f"Proveedor: {datos['proveedor']}")
        print(f"Precio: {datos['precio']:.2f}")
        print(f"Cajas: {datos['cajas']}")
        print(f"Stock total: {stock_total}")
        print()

```

De ambos ejercicios también pongo respuesta de ustedes

Ejercicio 7:

Escriba un programa que ingrese datos de patentes. El usuario deberá ingresar la cantidad de datos a ingresar. Los datos de patente son: Nombre, Apellido, fecha de Inscripción, patente.

```

import re
from datetime import datetime

def validar_fecha(fecha):
    try:
        datetime.strptime(fecha, "%d/%m/%Y")
        return True
    except ValueError:
        return False

def validar_texto(texto):
    return bool(texto.strip()) and all(c.isalpha() or c.isspace() for c in texto)

def validar_patente(patente, tipo):
    if tipo == "A":
        patron = r"^[A-Z]{2}\d{3}[A-Z]{2}$" # LL XXX LL
    elif tipo == "B":
        patron = r"^[A-Z]{3}\d{3}$" # LLL XXX
    else:
        return False

```

```

        return re.match(patron, patente) is not None

def guardar_datos(patentes, archivo):
    with open(archivo, 'w') as file:
        for i, patente in enumerate(patentes, start=1):
            file.write(f"Patente {i}:\n")
            file.write(f"Nombre: {patente['Nombre']}\n")
            file.write(f"Apellido: {patente['Apellido']}\n")
            file.write(f"Fecha de Inscripción: {patente['Fecha de
Inscripción']}\n")
            file.write(f"Patente: {patente['Patente']} (Tipo
{patente['Tipo']})\n")
            file.write("-" * 30 + "\n")
        print(f"Datos guardados en {archivo}")
        input("\nPresione ENTER para cerrar...")

def main():
    while True:
        try:
            cantidad = int(input("¿Cuántas patentes deseas ingresar? "))
            if cantidad > 0:
                break
            else:
                print("La cantidad debe ser un número entero positivo.")
        except ValueError:
            print("Entrada inválida. Debe ingresar un número entero.")
    patentes = []

    for _ in range(cantidad):
        print(f"\nIngresando datos para patente {_ + 1}:")
        while True:
            nombre = input("Nombre: ").strip()
            if validar_texto(nombre):
                break
            else:
                print("Nombre inválido. Debe ser solo texto y no puede estar
vacío.")
        while True:
            apellido = input("Apellido: ").strip()
            if validar_texto(apellido):
                break
            else:
                print("Apellido inválido. Debe ser solo texto y no puede estar
vacío.")
        while True:
            fecha = input("Fecha de Inscripción (dd/mm/aaaa): ").strip()
            if validar_fecha(fecha):
                break
            else:
                print("Fecha inválida. Debe estar en el formato dd/mm/aaaa.")
        while True:
            tipo = input("\nTipo de patente (A o B)\nDebe seguir el formato

```

```

'LLXXXLL' para Tipo A o 'LLLXXX' para Tipo B: ").strip().upper()
    if tipo in ["A", "B"]:
        break
    else:
        print("Tipo inválido. Debe ser 'A' o 'B'.")

while True:
    patente = input(f"Patente TIPO {tipo}: ").strip().upper()
    if validar_patente(patente, tipo):
        break
    else:
        print(f"Patente inválida para el Tipo {tipo}. "
              "Debe seguir el formato 'LLXXXLL' para Tipo A o 'LLLXXX'
para Tipo B.\nSin ESPACIOS ni CARACTERES ESPECIALES.")

    patente_data = {
        'Nombre': nombre,
        'Apellido': apellido,
        'Fecha de Inscripción': fecha,
        'Patente': patente,
        'Tipo': tipo
    }

    patentes.append(patente_data)
    print("\nDatos de patente ingresados exitosamente.")

guardar_datos(patentes, 'datos_patentes.txt')
main()

# Este código genera un archivo .txt
# Donde el contenido de los datos insertados serán datos guardados.

```

Ejercicio 8:

Realizar un programa que imprima los apellidos de las personas que no pagarán el impuesto de patente. Tener en cuenta que los autos cuyas patentes empiezan con **R**, **S** y **T** no pagan impuestos.

```

def cargar_datos(archivo):
    patentes_exentas = []
    try:
        with open(archivo, 'r') as file:
            lines = file.readlines()
            i = 0
            while i < len(lines):
                if lines[i].startswith("Patente"):
                    try:
                        patente = lines[i + 4].strip().split(": ")[1].strip()

```

```

        if patente and patente[0] in ('R', 'S', 'T'):
            apellido = lines[i + 2].strip().split(": ")
[1].strip()

            patentes_exentas.append(apellido)
        except (IndexError, ValueError) as e:
            print(f"Error al procesar los datos de la patente a
partir de la línea {i}. Error: {e}")
            i += 6
        else:
            i += 1
    except Exception as e:
        print(f"Error al leer el archivo: {e}")
    return patentes_exentas

def imprimir_exentos(patentes_exentas):
    print("Apellidos de las personas que no pagarán el impuesto de patente:")
    for apellido in patentes_exentas:
        print(apellido)

def main():
    archivo = 'datos_patentes.txt'
    patentes_exentas = cargar_datos(archivo)
    if not patentes_exentas:
        print("No se cargaron datos o el archivo está vacío.")
        return
    imprimir_exentos(patentes_exentas)
main()
input("\nPresione ENTER para cerrar...")

# Tener en cuenta que el código NO funcionara si
# no se ha insertado valores en la práctica 7.
# Debe detectar "datos_patentes.txt" para leerlos.

```